While the `npm run test:unit` command provides a report directly in your terminal (and some local test runners, like Vitest, often used in similar setups, might offer a web-based UI on a localhost port for interactive debugging), there are also excellent online test playgrounds that allow you to explore testing concepts visually.

One such example is the Testing Playground available at https://testing-playground.com/. You can open this URL in your browser to visualize and experiment with testing libraries (such as Testing Library, often used for React components) and interact with the HTML markup.

2.5.7 Common challenges and solutions

As you work through this project, be aware of these common problems and their solutions:

- *API configuration*—If you are having problems with the chat agent not responding, it might be a problem with the provider. First, ensure your .env file is configured with your Google Gemini API key, as Gemini is free for the examples in this book. If you're using OpenAI, note that it requires a credit card and sufficient available credits for API key usage. Double-check your OpenAI account for these details.
- *Handling CORS problems*—Since the backend (Express.js) is served on `localhost:3000` and the frontend (React) is on a different port, you may encounter CORS problems. To fix this problem, make sure that the following line is present in the server.js code:

```
app.use(cors());
```

- *General troubleshooting tips*—If you're experiencing problems, you can also check your browser's console for error messages and verify that both your frontend and backend servers are running.

2.6 Assessing the app's first iteration

Now the chat web application works reasonably well and can be used as a personal AI assistant or LLM agent, but it's rather simple. As you interact with it more, you will notice that it's very bare bones and hasn't got much functionality other than the main feature. Some fundamental characteristics are still missing:

- *Memory and context*—The application lacks the ability to remember or record conversation history. This means it forgets previous interactions upon reloading, leading to a lack of context and repetitive responses. As I mentioned earlier, this is because the message history isn't sent with the request to the backend.
- *UI performance*—The UI experiences loading delays because responses aren't streamed from the backend. They are sent in their entirety after the LLM has finished generating the full response.